

**PRAKTIKUM
SISTEM BASIS DATA**

Nama	Alexander Christhian	No. Modul	5
NPM	2306267025	Tipe	TP

PART 1

1. API adalah sebuah protokol yang membantu untuk sebuah software application atau web untuk dapat berinteraksi dengan software lainnya atau memiliki fitur tertentu. Dengan API, software dapat memiliki akses untuk mengakses dan berinteraksi dengan software lainnya tanpa harus melakukan analisa detail tentang software lainnya tersebut (seperti semacam jembatan). Terdapat beberapa istilah mengenai API :
 - a. Endpoint : Semacam URL atau alamat dalam API yang menjadi semacam alat komunikasi untuk client dan server. Setiap endpoint akan merepresentasikan function - function tertentu.
 - b. Method : Biasanya berbentuk HTTP dimana method ini akan menspecify tipe dari operation yang akan dilakukan seperti GET (menerima data dari server), POST (mengirim data ke server), PUT (mengupdate data), dll.
 - c. Body : Bagian dari API yang terdapat data yang akan dikirimkan ke server. Biasanya digunakan dalam POST dan PUT dalam bentuk JSON atau XML.
 - d. Params dan Query : Kedua ini digunakan untuk filter atau menspecify repsons API. Params atau path parameter akan mendefine dalam URL mengenai resource yang spesifik. Query akan diappend ke URL untuk memodifikasi requestnya.
 - e. Headers : Memberikan metadata tambahan untuk API request dan respons.
 - f. Status codes : Ini akan memberikan satuts yang mendeskripsikan hasil dari requestnya seperti 200 OK, 400 Bad Request, 500 Internal Server Error, dll.

Referensi:

- DZone "Using Query Parameters and Headers in REST API Design," dzone.com, 2023. [Online]. Available: <https://dzone.com/articles/using-query-parameters-and-headers-in-rest-api-des>. [Accessed 10 Mar. 2025].
- OneLoop "Understanding Query Params, Headers, and Body Params in FastAPI: When to

Use Each," oneloop.ai, 2024. [Online]. Available:

<https://www.oneloop.ai/blog/understanding-query-params-headers-and-body-params-in-fastapi-when-to-use-each>. [Accessed 10 Mar. 2025].

- Moesif "REST API Design Best Practices for Parameters and Query String Usage," moesif.com, 2022. [Online]. Available: <https://www.moesif.com/blog/technical/api-design/REST-API-Design-Best-Practices-for-Parameters-and-Query-String-Usage/>. [Accessed 10 Mar. 2025].

2. Diagram tersebut adalah diagram yang menjelaskan mengenai architecture untuk client-server dalam sebuah website. Semua ini akan menjelaskan mengenai frontend, backend, dan komponen dari databasenya. Penjelasan untuk masing - masing bagian :

a. Client dengan frontend :

Client akan menginisiasi request ke frontend dan frontend akan merespons dengan memberikan static files seperti .html, .css, dan .js untuk membentuk web tersebut.

b. Client request dengan backend :

Client akan merequest ke backend untuk datanya melalui sebuah API. Requestnya biasanya akan berbentuk HTTP request dengan method - methodnya dalam bentuk JSON.

c. Proses backend :

Backend akan melalui beberapa layer. Main akan bekerja sebagai entry point utama untuk handle API request. Route akan menentukan controller mana yang akan handle request tersebut. Controller adalah logic utama yang mengatur dan memproses data tersebut. Repository akan bekerja sebagai jembatan antara controller dan databasenya, mengatur data access logic. Repository tersebut lalu akan mengambil dan mengupdate data dalam database.

d. Respons ke client :

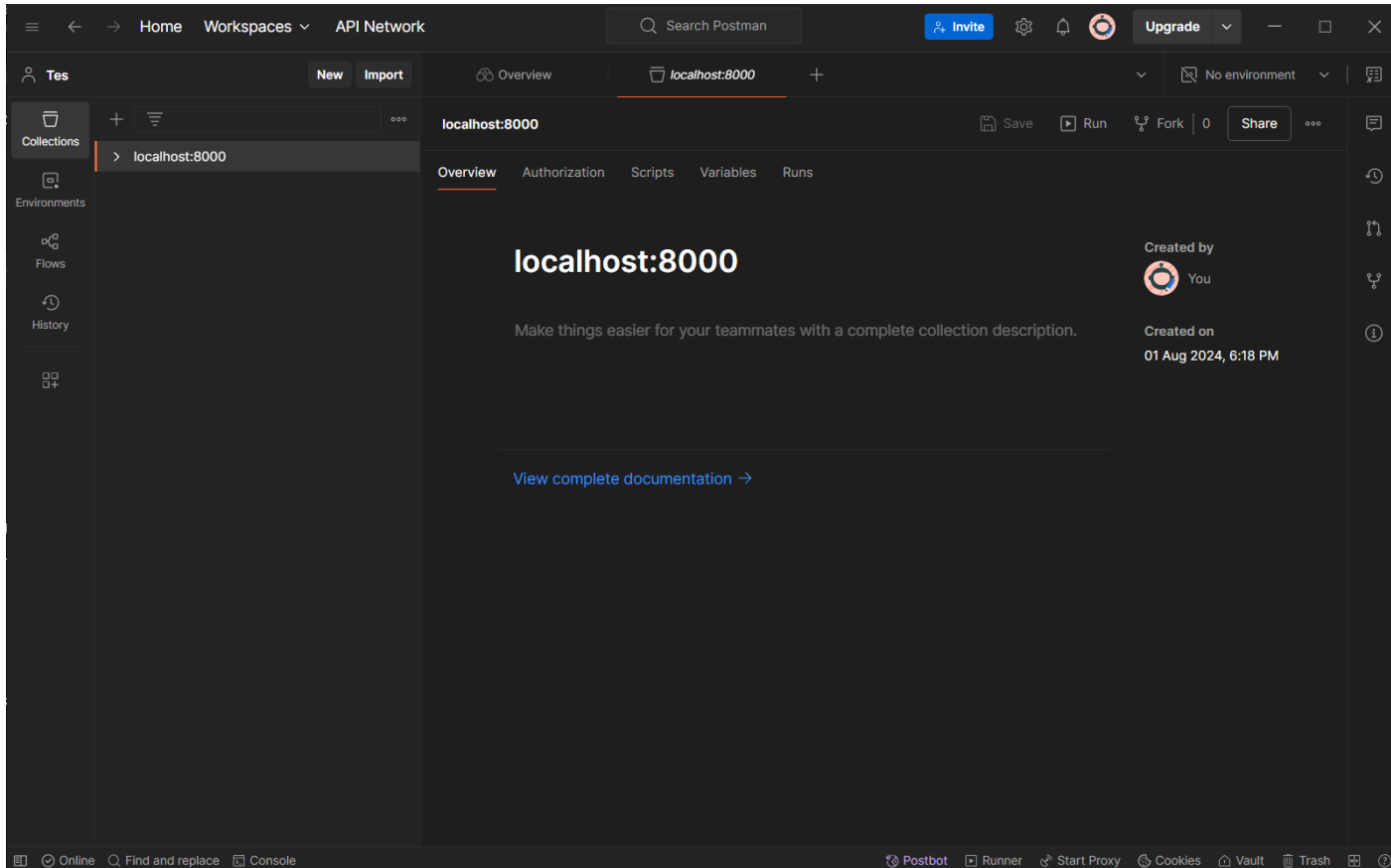
Setelah proses dari backend tersebut, akan diberikan return sebuah response dalam bentuk JSON format. Frontendnya lalu menerima JSON dan mengupdate UI tersebut.

Referensi:

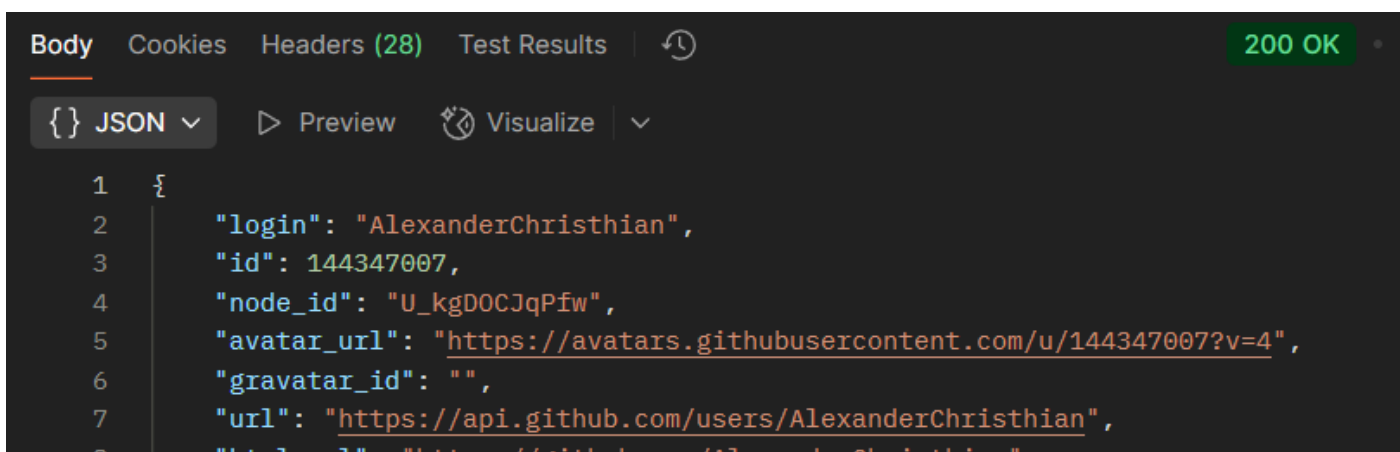
- Red Hat, "5 essential patterns of software architecture," redhat.com, 2021. [Online]. Available: <https://www.redhat.com/en/blog/5-essential-patterns-software-architecture>. [Accessed 8 Mar. 2025].

PART 2

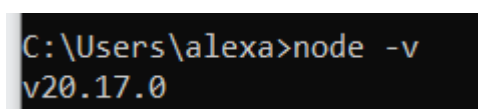
1. Download & install postman :



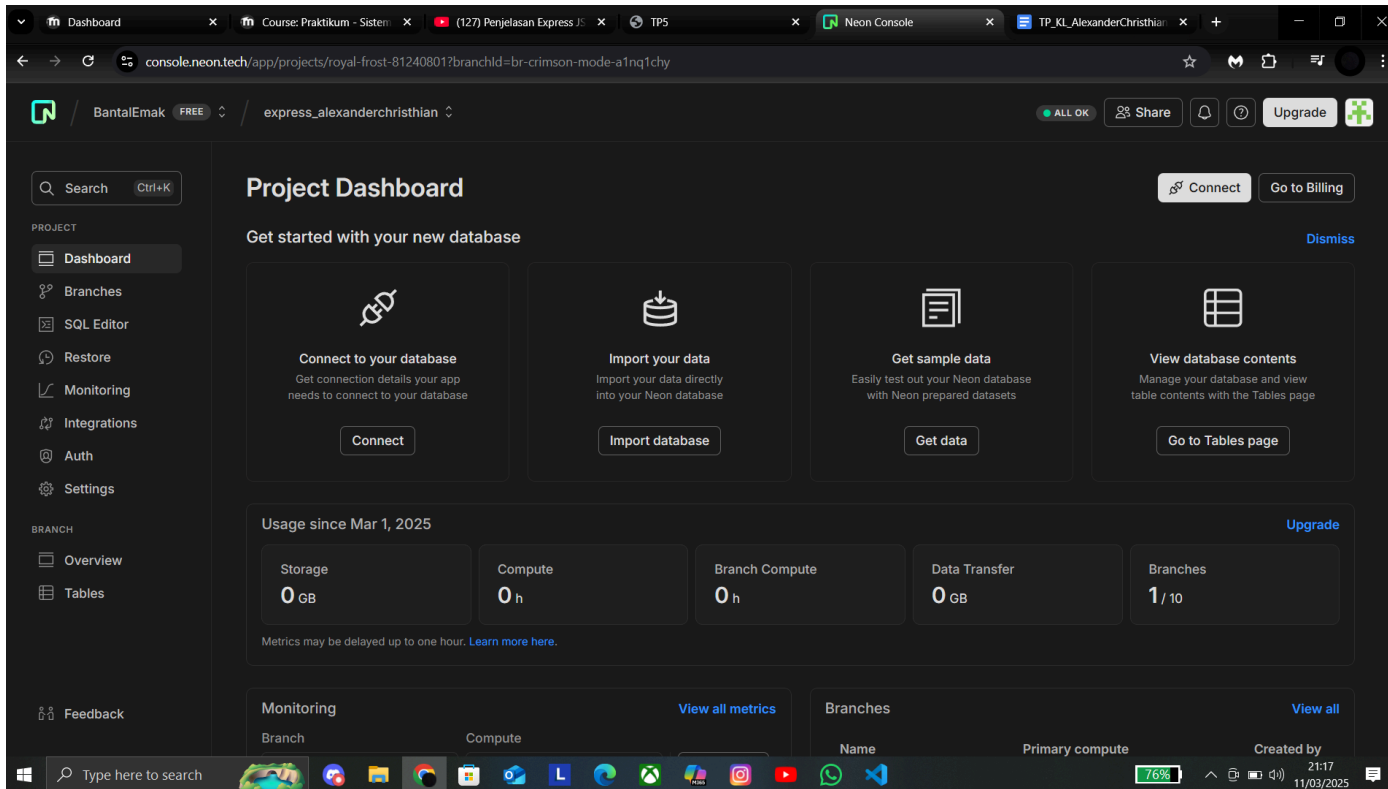
2. Request APInya :



4. Install node.js



5. Buat database :



6. Jalankan query :

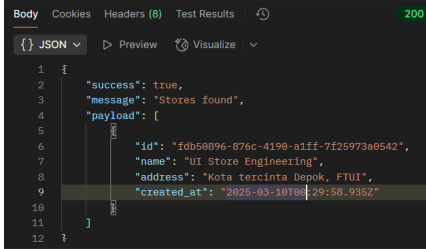
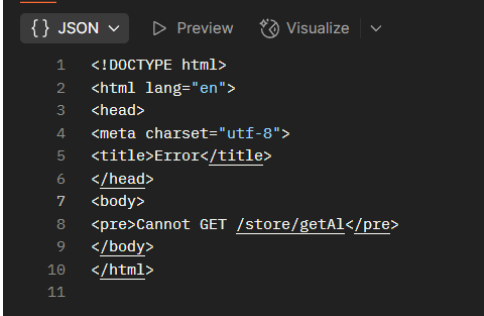
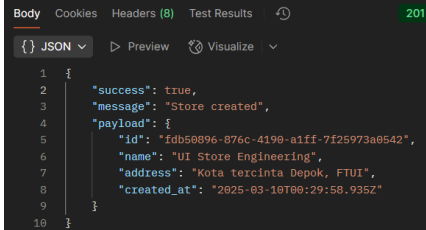
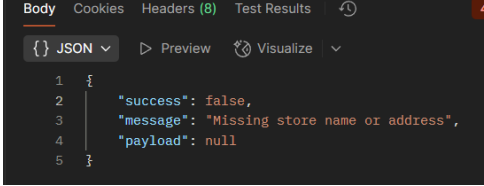
```

1 CREATE EXTENSION IF NOT EXISTS "uuid-oss";
2 CREATE TABLE IF NOT EXISTS stores(
3   id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
4   name VARCHAR(255) NOT NULL,
5   address VARCHAR(255) NOT NULL,
6   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
7 );|

```

8. Screenshot Response :

Method	Endpoint	Success	Failed
--------	----------	---------	--------

GET	/store/getAll	 <pre> 1 { 2 "success": true, 3 "message": "Stores found", 4 "payload": [5 { 6 "id": "fdb50896-876c-4190-a1ff-7f25973a0542", 7 "name": "UI Store Engineering", 8 "address": "Kota tercinta Depok, FTUI", 9 "created_at": "2025-03-10T00:29:58.935Z" 10 } 11] 12 }</pre>	 <pre> 1 <!DOCTYPE html> 2 <html lang="en"> 3 <head> 4 <meta charset="utf-8"> 5 <title>Error</title> 6 </head> 7 <body> 8 <pre>Cannot GET /store/getAll</pre> 9 </body> 10 </html> 11</pre>
POST	/store/create	 <pre> 1 { 2 "success": true, 3 "message": "Store created", 4 "payload": { 5 "id": "fdb50896-876c-4190-a1ff-7f25973a0542", 6 "name": "UI Store Engineering", 7 "address": "Kota tercinta Depok, FTUI", 8 "created_at": "2025-03-10T00:29:58.935Z" 9 } 10 }</pre>	 <pre> 1 { 2 "success": false, 3 "message": "Missing store name or address", 4 "payload": null 5 }</pre>